Python - Loop Sets

Loop Through Set Items

Looping through set items in Python refers to iterating over each element in a set. We can later perform required operations on each item. These operation includes list printing elements, conditional operations, filtering elements etc.

Unlike lists and tuples, sets are unordered collections, so the elements will be accessed in an arbitrary order. You can use a **for** loop to iterate through the items in a set.

Loop Through Set Items with For Loop

A for loop in Python is used to iterate over a sequence (like a list, tuple, dictionary, string, or range) or any other iterable object. It allows you to execute a block of code repeatedly for each item in the sequence.

In a for loop, you can access each item in a sequence using a variable, allowing you to perform operations or logic based on that item's value. We can loop through set items using for loop by iterating over each item in the set.

Syntax

Following is the basic syntax to loop through items in a set using a for loop in Python −

```
for item in set:
   # Code block to execute
```

Example

In the following example, we are using a for loop to iterate through each element in the set "my_set" and retrieving each element −

```
# Defining a set with multiple elements
my_set = {25, 12, 10, -21, 10, 100}

# Loop through each item in the set
for item in my_set:
   # Performing operations on each element
   print("Item:", item)
```

Output

Following is the output of the above code −

```
Item: 100
Item: 25
Item: 10
Item: -21
Item: 12
```

Loop Through Set Items with While Loop

A while loop in Python is used to repeatedly execute a block of code as long as a specified condition evaluates to "True".

We can loop through set items using a while loop by converting the set to an iterator and then iterating over each element until the iterator reaches end of the set.

> *An iterator is an object that allows you to traverse through all the elements of a collection (such as a list, tuple, set, or dictionary) one element at a time.*

Example

In the below example, we are iterating through a set using an iterator and a while loop. The "try" block retrieves and prints each item, while the "except StopIteration" block breaks the loop when there are no more items to fetch −

```
# Defining a set with multiple elements
my_set = {1, 2, 3, 4, 5}

# Converting the set to an iterator
set_iterator = iter(my_set)

# Looping through each item in the set using a while loop
while True:
   try:
      # Getting the next item from the iterator
      item = next(set_iterator)
      # Performing operations on each element
      print("Item:", item)
   except StopIteration:
      # If StopIteration is raised, break from the loop
      break
```

Output

Output of the above code is as follows –

```
Item: 1
Item: 2
Item: 3
Item: 4
Item: 5
```

Iterate using Set Comprehension

A set comprehension in Python is a concise way to create sets by iterating over an iterable and optionally applying a condition. It is used to generate sets using a syntax similar to list comprehensions but results in a set, ensuring all elements are unique and unordered.

We can iterate using set comprehension by defining a set comprehension expression within curly braces {} and specifying the iteration and condition logic within the expression. Following is the syntax –

```
result_set = {expression for item in iterable if condition}
```

Where,

- **expression –** It is an expression to evaluate for each item in the iterable.
- **item –** It is a variable representing each element in the iterable.
- **iterable –** It is a collection to iterate over (e.g., list, tuple, set).
- **condition –** It is optional condition to filter elements included in the resulting set.

Example

In this example, we are using a set comprehension to generate a set containing squares of even numbers from the original list "numbers" –

```
# Original list
numbers = [1, 2, 3, 4, 5]

# Set comprehension to create a set of squares of even numbers
squares_of_evens = {x**2 for x in numbers if x % 2 == 0}
# Print the resulting set
print(squares_of_evens)
```

Output

We get the output as shown below –

```
{16, 4}
```

Iterate through a Set Using the enumerate() Function

The enumerate() function in Python is used to iterate over an iterable object while also providing the index of each element.

We can iterate through a set using the enumerate() function by converting the set into a list and then applying enumerate() to iterate over the elements along with their index positions. Following is the syntax –

```
for index, item in enumerate(list(my_set)):
   # Your code here
```

Example

In the following example, we are first converting a set into a list. Then, we iterate through the list using a for loop with enumerate() function, retrieving each item along with its index −

```
# Converting the set into a list
my_set = {1, 2, 3, 4, 5}
set_list = list(my_set)

# Iterating through the list
for index, item in enumerate(set_list):
   print("Index:", index, "Item:", item)
```

Output

The output produced is as shown below −

```
Index: 0 Item: 1
Index: 1 Item: 2
Index: 2 Item: 3
Index: 3 Item: 4
Index: 4 Item: 5
```

Loop Through Set Items with add() Method

The add() method in Python is used to add a single element to a set. If the element is already present in the set, the set remains unchanged.

> *We cannot directly loop through set items using the add() method because add() is used specifically to add individual elements to a set, not to iterate over existing elements.*

To loop through set items, we use methods like a for loop or set comprehension.

Example

In this example, we loop through a sequence of numbers and add each number to the set using the add() method. The loop iterates over existing elements, while add() method adds new elements to the set −

```
# Creating an empty set
my_set = set()

# Looping through a sequence and adding elements to the set
for i in range(5):
   my_set.add(i)
print(my_set)
```

It will produce the following output −

```
{0, 1, 2, 3, 4}
```

Git Tutorial

Ethical Hacking Tutorial

Docker Tutorial

Kubernetes Tutorial

DSA Tutorial

Spring Boot Tutorial

SDLC Tutorial

Unix Tutorial

CERTIFICATIONS

Business Analytics Certification

Java & Spring Boot Advanced Certification

Data Science Advanced Certification

Cloud Computing And DevOps

Advanced Certification In Business Analytics

Artificial Intelligence And Machine Learning

DevOps Certification

Game Development Certification

Front-End Developer Certification

AWS Certification Training

Python Programming Certification

COMPILERS & EDITORS

Online Java Compiler

Online Python Compiler

Online C++ Compiler

Online C# Compiler

Online PHP Compiler

Online MATLAB Compiler

Online Bash Terminal

Online SQL Compiler

Online Html Editor